

elements, which are spaces, we get the location where the space begins by using the `start()` function. We are looking for potential words between those spaces. It will be useful to have another list that records the locations of characters that come just after a space; these will be the locations of the first character of each potential word. We'll call that list `spacestarts_affine`, since in technical terms, this new list is an affine transformation of the `spacestarts` list. *Affine* is often used to refer to linear transformations, such as adding 1 to each location, which we'll do here. We'll also sort this list:

```
spacestarts_affine = [ss+1 for ss in spacestarts]
spacestarts_affine.sort()
```

Next, we can get all the substrings that are between two spaces:

```
between_spaces = [(spacestarts[k] + 1, spacestarts[k + 1]) for k in
range(0, len(spacestarts) - 1 )]
```

The variable we're creating here is called `between_spaces`, and it's a list of tuples of the form (*<location of beginning of substring>*, *<location of end of substring>*), like (17, 23). The way we get these tuples is through a list comprehension. This list comprehension iterates over k . In this case, k takes on the values of integers between 0 and one less than the length of the `spacestarts` list. For each k , we will generate one tuple. The first element of the tuple is `spacestarts[k]+1`, which is one position after the location of each space. The second element of the tuple is `spacestarts[k+1]`, which is the location of the next space in the text. This way, our final output contains tuples that indicate the beginning and end of each substring between spaces.

Now, consider all of the potential words that are between spaces, and find the ones that are not valid (not in our word list):

```
between_spaces_notvalid = [loc for loc in between_spaces if \text[loc[0]:loc[1]]
not in word_list]
```

Looking at `between_spaces_notvalid`, we can see that it's a list of the locations of all invalid potential words in our text:

```
[(4, 16), (24, 30), (31, 34), (35, 45), (46, 48), (49, 54), (55, 68), (69, 71),
(72, 78), (79, 81), (82, 84), (85, 90), (91, 92), (93, 105), (106, 107), (108,
111), (112, 119), (120, 123)]
```
